Valex API

Documentación técnica: base de datos y API REST

Proyecto: Creaciones Valery's — tienda de amigurumis personalizados

Dato	Valor
Versión del documento	1.0
Fecha	22 de octubre de 2026
Esquema de base de datos	Migración `001_initial_schema`
Tecnologías	Next.js 16 · PostgreSQL 18 · TypeScript · Zod · JWT (jose)
Repositorio	`valex-api/`

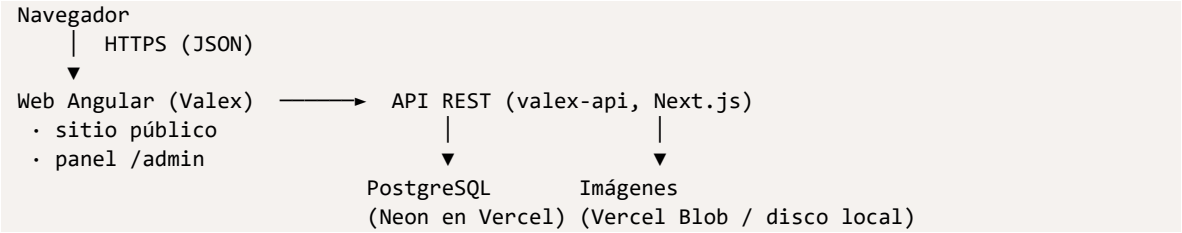
Contenido

1. Arquitectura 3
1.1 Tecnologías 3
1.2 Decisiones de diseño 3
2. Base de datos 4
2.1 Diagrama de relaciones 4
2.2 Tabla products 4
2.3 Tabla product_details 4
2.4 Tabla site_settings 5
2.5 Tabla content_items 5
2.6 Tabla schema_migrations 6
2.7 Funciones, triggers y migraciones 6
2.8 Datos iniciales 6
3. API REST: convenciones 7
3.1 Formato de error 7
3.2 Resumen de endpoints 7
4. API REST: referencia de endpoints 9
4.1 Autenticación 9
4.2 Información del negocio 9
4.3 Contenido de la portada 11
4.4 Productos 11
4.5 Imágenes 15
5. Seguridad 16
6. Configuración y despliegue 17
6.1 Variables de entorno 17
6.2 Scripts 17
6.3 Despliegue en Vercel 17

Si el índice aparece vacío, en Word haz clic derecho sobre él y elige «Actualizar campo».

1. Arquitectura

El sistema tiene tres piezas. La web Angular (sitio público y panel de administración) consume la API REST, y la API es la única que accede a la base de datos y al almacenamiento de imágenes.



1.1 Tecnologías

Capa	Tecnología	Uso
API	Next.js 16 (route handlers)	Endpoints REST desplegables como funciones serverless en Vercel
Validación	Zod 4	Valida y normaliza cada cuerpo de petición
Autenticación	JWT HS256 (jose)	Token de administrador con vigencia de 8 horas
Base de datos	PostgreSQL 18	Local con `embedded-postgres`; en producción, Neon
Acceso a datos	postgres.js	SQL parametrizado (sin riesgo de inyección SQL)
Imágenes	Vercel Blob / disco	La base de datos guarda solo la URL de la imagen

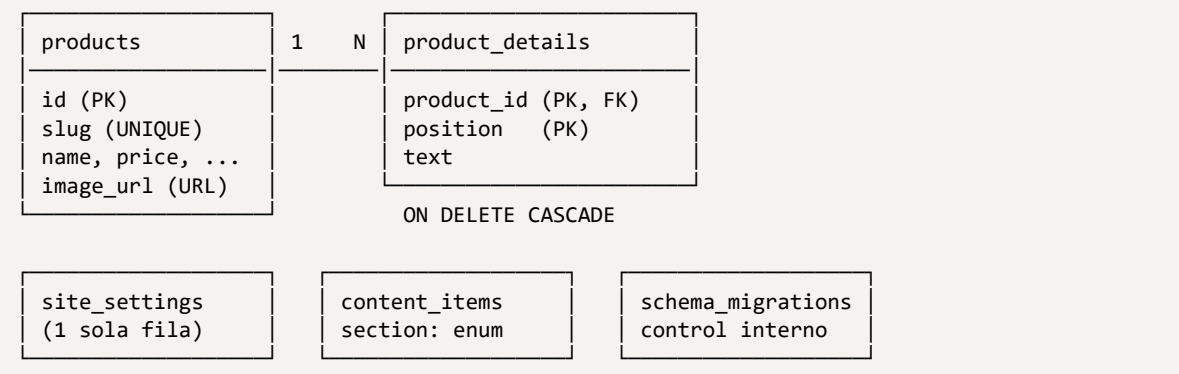
1.2 Decisiones de diseño

- Imágenes por URL. Las tablas guardan rutas como `/images/doll-1.jpg` o URLs absolutas de Blob, nunca el archivo. Así la base de datos se mantiene pequeña y las imágenes se sirven desde CDN.
- Patrón repositorio: las rutas de la API llaman a una interfaz `Repository`; solo `PostgresRepository` conoce SQL.
- Migraciones versionadas: cada cambio de esquema es una migración nueva, registrada en `schema_migrations`. La API las aplica sola en el primer arranque.
- Precio opcional: `price = null` significa «precio a consultar». El negocio no publica precios fijos.

2. Base de datos

Motor: PostgreSQL 18. Esquema `public`. Todas las tablas usan claves primarias enteras generadas por la base (`identity`), salvo `site_settings`, que tiene una única fila.

2.1 Diagrama de relaciones



2.2 Tabla products

Catálogo de amigurumis. Índice `products_catalog_idx (is_visible, sort_order, id)` para la consulta del catálogo público.

Columna	Tipo	Nulo	Por defecto	Restricciones	Descripción
`id`	integer	No	identity	PK	Identificador
`slug`	varchar(140)	No	—	UNIQUE; formato `a-z0-9` con guiones	Identificador para la URL pública
`name`	varchar(120)	No	—	Mínimo 2 caracteres	Nombre del modelo
`description`	varchar(2000)	No	vacío	—	Descripción larga
`image_url`	varchar(500)	No	—	—	URL de la imagen (no el archivo)
`image_alt`	varchar(300)	No	vacío	—	Texto alternativo (accesibilidad)
`price`	numeric(10,2)	Sí	null	`price >= 0`	Precio en soles; null = a consultar
`size_info`	varchar(200)	Sí	null	—	Tamaño, por ejemplo «25 cm aprox.»
`is_visible`	boolean	No	true	—	Si aparece en el sitio público
`sort_order`	integer	No	0	Entre 0 y 10000	Orden en el catálogo
`created_at`	timestamptz	No	now()	—	Fecha de creación
`updated_at`	timestamptz	No	now()	Trigger	Última modificación

2.3 Tabla product_details

Detalles personalizados de cada producto, en orden. Relación 1:N con `products`; al borrar un producto se borran sus detalles (`ON DELETE CASCADE`).

Columna	Tipo	Nulo	Por defecto	Restricciones	Descripción
product_id	integer	No	—	PK; FK → products.id	Producto al que pertenece
position	smallint	No	—	PK; position >= 0	Orden del detalle
text	varchar(80)	No	—	No vacío	Texto, por ejemplo «Lentes dorados»

2.4 Tabla site_settings

Información del negocio que se muestra en todo el sitio. Tiene exactamente una fila: CHECK (id = 1).

Columna	Tipo	Nulo	Por defecto	Restricciones	Descripción
id	smallint	No	1	PK; id = 1	Fila única
name	varchar(80)	No	—	No vacío	Nombre del negocio
tagline	varchar(160)	No	vacío	—	Eslogan
whatsapp_number	varchar(15)	No	—	Solo dígitos, 8 a 15	Número para enlaces wa.me
whatsapp_display	varchar(30)	No	—	—	Número tal como se muestra
default_message	varchar(300)	No	vacío	—	Mensaje predeterminado de WhatsApp
facebook_url	varchar(500)	No	vacío	—	Página de Facebook
instagram_handle	varchar(60)	No	vacío	—	Usuario de Instagram
instagram_url	varchar(500)	No	vacío	—	Perfil de Instagram
location	varchar(120)	No	vacío	—	Zona («San Juan de Miraflores, Lima»)
address	varchar(200)	No	vacío	—	Dirección exacta (se enlaza a Google Maps)
logo_url	varchar(500)	No	—	—	URL del logo
hero_image_url	varchar(500)	No	—	—	URL de la imagen de portada
updated_at	timestamptz	No	now()	Trigger	Última modificación

2.5 Tabla content_items

Bloques de texto de la portada. La columna section usa el tipo enumerado content_section (highlight, step, occasion). Índice content_items_section_idx (section, sort_order).

Columna	Tipo	Nulo	Por defecto	Restricciones	Descripción
id	integer	No	identity	PK	Identificador
section	content_section	No	—	Enum	Ventaja, paso de compra u ocasión
icon	varchar(40)	No	—	—	Nombre del ícono del frontend

Columna	Tipo	Nulo	Por defecto	Restricciones	Descripción
title	varchar(120)	No	—	—	Título
text	varchar(300)	No	vacío	—	Texto (vacío en ocasiones)
sort_order	integer	No	0	—	Orden dentro de su sección

2.6 Tabla schema_migrations

Columna	Tipo	Nulo	Por defecto	Restricciones	Descripción
id	varchar(100)	No	—	PK	Nombre de la migración aplicada
applied_at	timestamptz	No	now()	—	Cuándo se aplicó

2.7 Funciones, triggers y migraciones

- Función `set_updated_at()` (plpgsql): pone `updated_at = now()`. Se ejecuta con los triggers `products_updated_at` y `site_settings_updated_at` antes de cada `UPDATE`.
- Las migraciones viven en `src/lib/db/migrations.ts` y se aplican en orden dentro de una transacción con `pg_advisory_xact_lock`, para que dos instancias no migren a la vez.
- Para cambiar el esquema se agrega una migración nueva (por ejemplo `002_...`). Nunca se edita una ya aplicada.

2.8 Datos iniciales

Con la base vacía, la API (o `npm run db:migrate`) carga: 1 fila de `site_settings`, 5 productos con 15 detalles y 13 bloques de contenido (3 ventajas, 4 pasos y 6 ocasiones). Los precios se dejan en `null`, porque el negocio no los publica.

3. API REST: convenciones

Aspecto	Detalle
URL base (local)	`http://localhost:5058`
Formato	JSON UTF-8 (`Content-Type: application/json; charset=utf-8`)
Autenticación	Cabecera `Authorization: Bearer <token>` obtenida con `POST /api/auth/login`
CORS	Solo los orígenes listados en `CORS_ORIGINS`
Caché	`Cache-Control: no-store` en respuestas JSON
Slug	Lo genera el servidor a partir del nombre; si se repite, se agrega `-2`, `-3`…
Precio	`price: null` = precio a consultar

3.1 Formato de error

Todos los errores responden con el mismo formato:

```
{
  "title": "Datos inválidos.",
  "status": 400,
  "errors": [{ "field": "name", "message": "El nombre debe tener al menos 2 caracteres." }]
}
```

Código	Significado
400	JSON mal formado, validación fallida o restricción de la base de datos
401	Sin token, token inválido o expirado; o credenciales incorrectas
404	El recurso no existe (o el producto está oculto, en rutas públicas)
409	Conflicto de unicidad en la base de datos
429	Demasiados intentos de login (más de 5 por minuto e IP)
500	Error interno (se registra en el servidor; no expone detalles)

3.2 Resumen de endpoints

Método	Ruta	Acceso	Descripción
POST	`/api/auth/login`	Público	Iniciar sesión y obtener un token
GET	`/api/site`	Público	Información del negocio
PUT	`/api/site`	Admin	Actualizar información del negocio
GET	`/api/content`	Público	Bloques de la portada
GET	`/api/products`	Público*	Catálogo (*ocultos solo para admin)
GET	`/api/products/{slug}`	Público	Detalle de un producto
POST	`/api/products`	Admin	Crear producto
GET	`/api/products/id/{id}`	Admin	Leer producto por id
PUT	`/api/products/id/{id}`	Admin	Editar producto
DELETE	`/api/products/id/{id}`	Admin	Borrar producto

Método	Ruta	Acceso	Descripción
POST	`/api/uploads`	Admin	Subir imagen
GET	`/uploads/{archivo}`	Público	Imagen subida (solo en desarrollo)

4. API REST: referencia de endpoints

4.1 Autenticación

POST /api/auth/login

Campo	Valor
Método	`POST`
Ruta	`/api/auth/login`
Acceso	Público
Descripción	Valida las credenciales del administrador y devuelve un JWT firmado con HS256, válido por 8 horas.

Cuerpo de la petición

```
{ "username": "admin", "password": "••••••••" }
```

Respuesta exitosa

```
200 OK
{
  "token": "eyJhbGciOiJIUzI1NiJ9...",
  "expiresAt": "2026-10-23T03:00:00.000Z",
  "username": "admin"
}
```

Errores posibles

Código	Cuándo
400	Faltan campos o el JSON es inválido
401	Usuario o contraseña incorrectos (mismo mensaje en ambos casos)
429	Más de 5 intentos por minuto desde la misma IP

4.2 Información del negocio

GET /api/site

Campo	Valor
Método	`GET`
Ruta	`/api/site`
Acceso	Público
Descripción	Devuelve la información del negocio que muestra el sitio (contacto, redes, imágenes).

Respuesta exitosa

```
200 OK
{
  "name": "Creaciones Valery's",
  "tagline": "Amigurumis únicos y personalizados",
  "whatsappNumber": "51947191624",
  "whatsappDisplay": "947 191 624",
  "defaultMessage": "¡Hola Creaciones Valery's! Quiero un amigurumi personalizado.",
  "facebookUrl": "https://www.facebook.com/valerystejido",
  "instagramHandle": "@valerystejido",
  "instagramUrl": "https://www.instagram.com/valerystejido/",
  "location": "San Juan de Miraflores, Lima",
  "address": "Jr. Alejandro Tirado 290, San Juan de Miraflores, Perú 15803",
  "logoUrl": "/images/logo.jpg",
  "heroImageUrl": "/images/dolls-group.jpg"
}
```

PUT /api/site

Campo	Valor
Método	PUT
Ruta	`/api/site`
Acceso	Administrador (requiere `Authorization: Bearer <token>`)
Descripción	Reemplaza la información del negocio. Si cambian el logo o la portada, borra la imagen subida anterior que ya no se use.

Cuerpo de la petición

```
{
  "name": "Creaciones Valery's",
  "tagline": "Amigurumis únicos y personalizados",
  "whatsappNumber": "51947191624",
  "whatsappDisplay": "947 191 624",
  "defaultMessage": "¡Hola Creaciones Valery's! Quiero un amigurumi personalizado.",
  "facebookUrl": "https://www.facebook.com/valerystejido",
  "instagramHandle": "@valerystejido",
  "instagramUrl": "https://www.instagram.com/valerystejido/",
  "location": "San Juan de Miraflores, Lima",
  "address": "Jr. Alejandro Tirado 290, San Juan de Miraflores, Perú 15803",
  "logoUrl": "/images/logo.jpg",
  "heroImageUrl": "/images/dolls-group.jpg"
}
```

Respuesta exitosa

```
200 OK — el objeto guardado (mismo formato que GET).
```

Errores posibles

Código	Cuándo
400	`whatsappNumber` no son 8-15 dígitos, URL inválida o campo obligatorio vacío
401	Falta el token, es inválido o expiró.

- Obligatorios: `name`, `whatsappNumber`, `whatsappDisplay`, `logoUrl`, `heroImageUrl`.

4.3 Contenido de la portada

GET /api/content

Campo	Valor
Método	`GET`
Ruta	`/api/content`
Acceso	Público
Descripción	Devuelve los bloques de la portada agrupados por sección y ordenados por `sortOrder`.

Respuesta exitosa

```
200 OK
{
  "highlights": [{ "id": 1, "icon": "hand", "title": "Hecho a mano", "text": "...", "sortOrder
": 1 }],
  "steps":      [{ "id": 4, "icon": "chat", "title": "Escríbenos", "text": "...", "sortOrder":
 1 }],
  "occasions":  [{ "id": 8, "icon": "cake", "title": "Cumpleaños", "text": "", "sortOrder": 1
}]
}
```

4.4 Productos

Objeto `Product` que devuelven todos los endpoints de productos:

```
{
  "id": 1,
  "slug": "con-su-nombre",
  "name": "Con su nombre",
  "description": "Con su uniforme de trabajo, sus lentes dorados...",
  "details": ["Lentes dorados", "Gafete personalizado", "Uniforme de trabajo"],
  "imageUrl": "/images/doll-1.jpg",
  "imageAlt": "Amigurumi de una mujer con lentes dorados...",
  "price": null,
  "sizeInfo": null,
  "isVisible": true,
  "sortOrder": 1,
  "createdAt": "2026-10-22T18:58:41.000Z",
  "updatedAt": "2026-10-22T18:58:41.000Z"
}
```

GET /api/products

Campo	Valor
Método	`GET`
Ruta	`/api/products`
Acceso	Público (los ocultos, solo para admin)
Descripción	Lista el catálogo ordenado por `sortOrder` y luego por `id`.

Parámetros:

Nombre	Ubicación	Descripción
includeHidden	Query	`true` para incluir productos ocultos. Solo funciona con token de admin; sin token se ignora.

Respuesta exitosa

```
200 OK — Product[]
```

GET /api/products/{slug}

Campo	Valor
Método	`GET`
Ruta	`/api/products/{slug}`
Acceso	Público
Descripción	Detalle de un producto visible. Lo usa la página `/productos/{slug}` de la web.

Parámetros:

Nombre	Ubicación	Descripción
slug	Ruta	Identificador del producto, por ejemplo `grupo-personalizado`

Respuesta exitosa

```
200 OK — Product
```

Errores posibles

Código	Cuándo
404	No existe o está oculto

POST /api/products

Campo	Valor
Método	`POST`
Ruta	`/api/products`
Acceso	Administrador (requiere `Authorization: Bearer <token>`)
Descripción	Crea un producto. El slug se genera a partir del nombre.

Cuerpo de la petición

```
{
  "name": "Grupo personalizado",
  "description": "Teje a todo tu equipo, tu familia o tu grupo de amigos.",
  "details": ["Varios personajes", "Cada uno personalizado"],
  "imageUrl": "/images/dolls-group.jpg",
  "imageAlt": "",
  "price": 120.5,
  "sizeInfo": "25 cm aprox.",
  "isVisible": true,
  "sortOrder": 5
}
```

Respuesta exitosa

```
201 Created — Product
```

Errores posibles

Código	Cuándo
400	Validación (ver reglas abajo)
401	Falta el token, es inválido o expiró.
409	Conflicto de slug en una carrera entre dos peticiones

- `name`: obligatorio, de 2 a 120 caracteres. `imageUrl`: obligatorio, máximo 500.
- `details`: máximo 12, de 80 caracteres cada uno; se quitan vacíos y repetidos.
- `price`: de 0 a 100000, o `null`. `sortOrder`: entero de 0 a 10000. `sizeInfo` vacío pasa a `null`.
- `imageAlt` vacío pasa a «Amigurumi {name}».

GET /api/products/id/{id}

Campo	Valor
Método	`GET`
Ruta	`/api/products/id/{id}`
Acceso	Administrador (requiere `Authorization: Bearer <token>`)
Descripción	Lee un producto por id, incluso si está oculto. Lo usa el formulario de edición del panel.

Parámetros:

Nombre	Ubicación	Descripción
id	Ruta	Entero positivo

Respuesta exitosa

```
200 OK — Product
```

Errores posibles

Código	Cuándo
400	Id inválido
401	Falta el token, es inválido o expiró.
404	No existe

PUT /api/products/id/{id}

Campo	Valor
Método	`PUT`
Ruta	`/api/products/id/{id}`
Acceso	Administrador (requiere `Authorization: Bearer <token>`)

Campo	Valor
Descripción	Reemplaza el producto completo (y sus detalles) en una transacción. Si cambió la imagen, borra la subida anterior que ya no se use.

Parámetros:

Nombre	Ubicación	Descripción
id	Ruta	Entero positivo

Cuerpo de la petición

```
Mismo cuerpo que POST /api/products.
```

Respuesta exitosa

```
200 OK — Product actualizado
```

Errores posibles

Código	Cuándo
400	Validación o id inválido
401	Falta el token, es inválido o expiró.
404	No existe

- Es un reemplazo completo: los campos omitidos toman su valor por defecto (por ejemplo, `isVisible = true`).

DELETE /api/products/id/{id}

Campo	Valor
Método	`DELETE`
Ruta	`/api/products/id/{id}`
Acceso	Administrador (requiere `Authorization: Bearer <token>`)
Descripción	Borra el producto. Sus detalles se eliminan en cascada y su imagen subida, si nadie más la usa.

Parámetros:

Nombre	Ubicación	Descripción
id	Ruta	Entero positivo

Respuesta exitosa

```
204 No Content
```

Errores posibles

Código	Cuándo
401	Falta el token, es inválido o expiró.
404	No existe

4.5 Imágenes

POST /api/uploads

Campo	Valor
Método	`POST`
Ruta	`/api/uploads`
Acceso	Administrador (requiere `Authorization: Bearer <token>`)
Descripción	Sube una imagen y devuelve su URL. Esa URL es la que se guarda luego en `imageUrl`, `logoUrl` o `heroImageUrl`.

Cuerpo de la petición

```
multipart/form-data
  file = <imagen JPG, PNG o WebP, máximo 4 MB>
```

Respuesta exitosa

```
201 Created
{ "url": "/uploads/9b1d...e2.jpg" }
```

Errores posibles

Código	Cuándo
400	Sin archivo, más de 4 MB o formato no soportado
401	Falta el token, es inválido o expiró.

- El formato se comprueba por la firma del archivo (magic bytes), no por la extensión ni por el tipo que declara el navegador.
- En producción (Vercel) la URL devuelta es absoluta, de Vercel Blob.

GET /uploads/{archivo}

Campo	Valor
Método	`GET`
Ruta	`/uploads/{archivo}`
Acceso	Público
Descripción	Sirve las imágenes subidas en desarrollo local (carpeta `data/uploads`). En producción no se usa.

Respuesta exitosa

```
200 OK — la imagen, con caché de un año.
```

Errores posibles

Código	Cuándo
404	No existe o la extensión no está permitida

5. Seguridad

- Rutas de administración protegidas con JWT HS256, firmado con `JWT_SECRET` de 32 caracteres o más, y vigencia de 8 horas.
- Comparación de credenciales en tiempo constante (`timingSafeEqual`) y el mismo mensaje para usuario o contraseña incorrectos.
- Límite de 5 intentos de login por minuto e IP. Es por instancia: en Vercel conviene un almacén compartido (por ejemplo, Upstash) para un límite estricto.
- SQL siempre parametrizado (postgres.js), validación con Zod y restricciones `CHECK`/`UNIQUE`/`FK` en la base de datos como segunda barrera.
- Subidas: tipo verificado por contenido, nombre aleatorio (UUID) y tamaño máximo de 4 MB.
- CORS restringido a los orígenes de `CORS_ORIGINS`.

6. Configuración y despliegue

6.1 Variables de entorno

Variable	Obligatoria	Descripción
`DATABASE_URL`	Sí	Conexión a PostgreSQL. Local: `postgres://valex:valex-local@localhost:5433/valex`. Vercel: la crea Neon.
`JWT_SECRET`	Sí	Clave para firmar tokens (32+ caracteres aleatorios)
`ADMIN_USERNAME`	Sí	Usuario del panel
`ADMIN_PASSWORD`	Sí	Contraseña del panel (distinta en producción)
`CORS_ORIGINS`	Sí	URLs de la web, separadas por comas
`BLOB_READ_WRITE_TOKEN`	En Vercel	Almacenamiento de imágenes; sin él, las subidas van a `data/uploads`

6.2 Scripts

Comando	Qué hace
`npm run db:start`	Arranca PostgreSQL 18 local en el puerto 5433 (lo crea la primera vez)
`npm run db:migrate`	Aplica migraciones pendientes y carga los datos iniciales si la base está vacía
`npm run db:info`	Lista tablas, columnas y cantidad de filas
`npm run db:reset -- --yes`	Borra todo y lo recrea (solo desarrollo)
`npm run dev`	API en `http://localhost:5058`
`npm run build`	Compilación de producción

6.3 Despliegue en Vercel

1. Crear el proyecto con la carpeta `valex-api` (framework: Next.js).
2. En Storage, agregar Neon (crea `DATABASE_URL`) y Blob (crea `BLOB_READ_WRITE_TOKEN`).
3. Definir `JWT_SECRET`, `ADMIN_USERNAME`, `ADMIN_PASSWORD` y `CORS_ORIGINS`.
4. Desplegar. La primera petición crea las tablas y los datos iniciales.
5. Poner la URL de la API en `Valex/src/environments/environment.prod.ts` y desplegar la web.

Nota: la conexión con Neon y Vercel Blob aún no se ha probado en un despliegue real. En local se validó con PostgreSQL 18 y almacenamiento en disco.